Chapter 2 : Query Processing and Optimization

Instructor: Melaku M.

Target Group: G3SE, Section A & B

Outline

- Overview
- Query Processing steps
- Query Decomposition
- Optimization Process
- Approaches to Query Optimization
- Transformation Rules
- Pipelining

Database Performance-Tuning Concepts

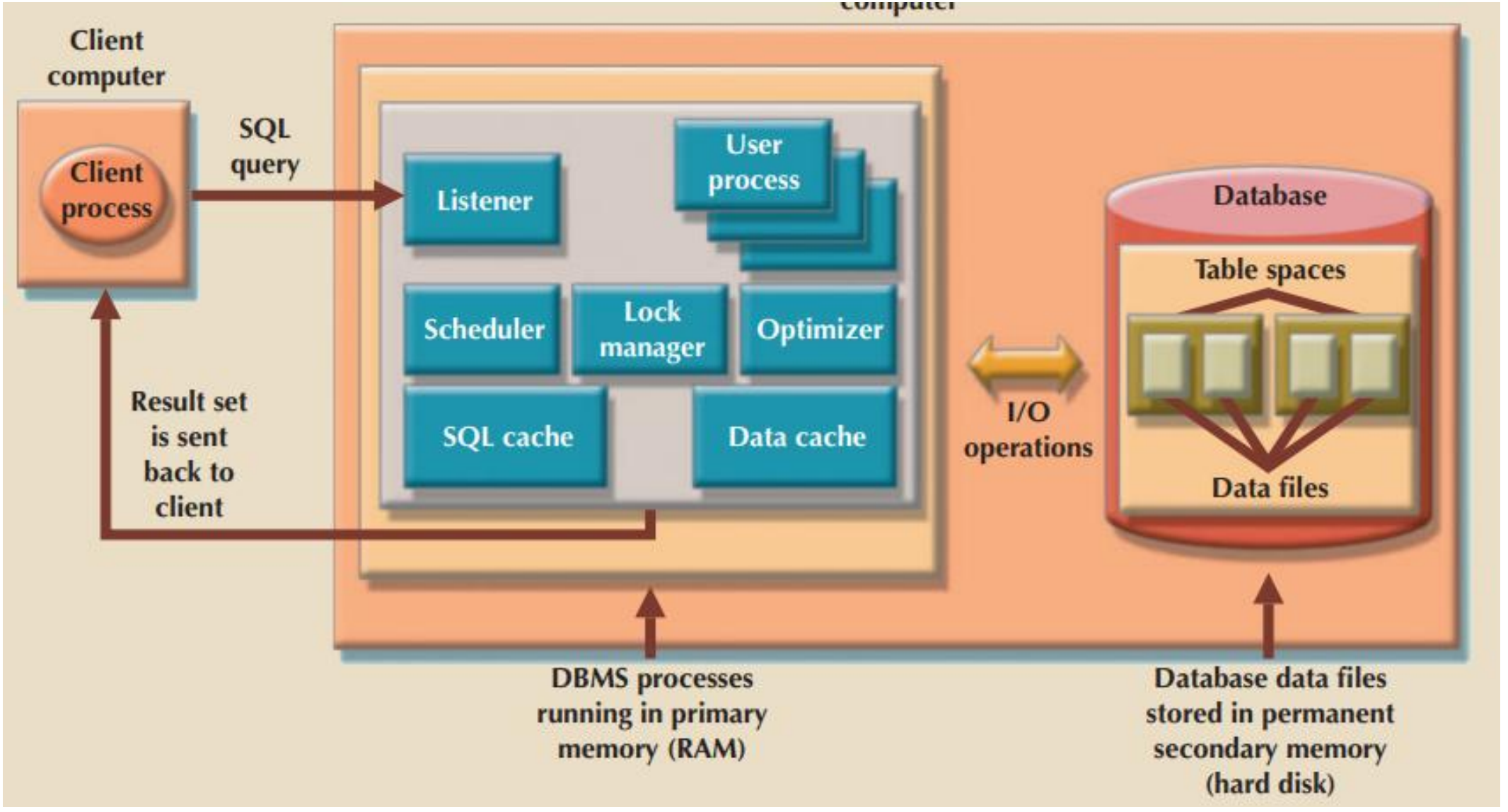
- One of the main functions of a database system is to provide timely answers to end users.
- End users interact with the DBMS through the use of queries to generate information, using the following sequence:

1. The end-user (client-end) application generates a query.
2. The query is sent to the DBMS (server end).
3. The DBMS (server end) executes the query.
4. The DBMS sends the resulting data set to the end-user (client-end) application.

Database Performance-Tuning Concepts

- **Database performance tuning** refers to a set of activities and procedures designed to reduce the **response time** of the database system—that is, to ensure that an end-user query is processed by the DBMS in the minimum amount of time.
- The time required by a query to return a result set depends on many factors.
- In general, the performance of a typical DBMS is constrained by three main factors: CPU processing power, available primary memory (RAM), and input/output (hard disk and network) throughput.

DBMS Architecture



Query processing

- **Query processing** refers to the **range of activities** involved in extracting data from a database.
- The activities include:
  - ✓ **translation** of **queries** in high-level database languages(SQL) into expressions/RA that can be used at the physical level of the file system,
  - ✓ a variety of **query-optimizing transformations**, and
  - ✓ actual evaluation/execution of queries.

The steps involved in query processing



query

parser and translator

relational-algebra expression

* Internal Representation

optimizer

execution plan

* Number of tuples in R.

evaluation engine

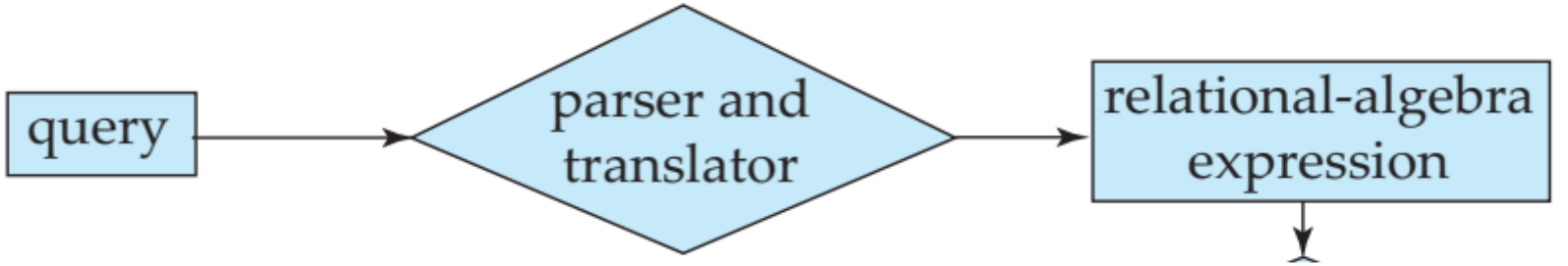
query output

data

statistics about data

Figure: Steps in query processing.

The steps involved in query processing



WHY?

SQL

A **declarative** expression of the query result

Say what you want, not how to get it!

Codd's theorem

RA

Operational description of a computation

Systems optimize and execute RA query plan!

The steps involved in query processing

- Before query processing can begin, the DBMS system must **translate** the query into a **usable form**.
- SQL is the best suitable choice for humans. But, it is not perfectly suitable for the system's internal representation of the query.
- **Relational algebra** is well suited for the internal representation of a query.
- Thus, the first action the system must take in query processing is to **translate** a given query into its **internal form** .
- This translation process is similar to the work performed by the **parser of a compiler.**

Cont'd

- In generating the **internal form** of the query, the **parser in system:**
  - ✓ checks **user's query** is syntactically correct.
  - ✓ Verify relations and attributes exist in database.
  - ✓ Verify operations are appropriate for object type and so on.
- The system constructs/creates a **parse-tree** representation of the query, which it then **translates** into a **relational-algebra expression**.
- **Note**: In query processing, the first step typically involves converting the **SQL** query into a **parse tree**.
  - ✓ Parsing: The SQL query is parsed to generate a parse tree or abstract syntax tree, which represents the syntactical/grammatical structure of the query.
  - ✓ Transforming to Relational Algebra: Once the parse tree is generated, it is then transformed into a relational algebra expression.

Cont'd

- Given a query, there are generally a variety of methods for computing the answer. For example, we have seen that, in SQL, a query could be expressed in several different ways.
- Each SQL query can itself be translated into a relational-algebra expression in one of **several ways**.
- Furthermore, the relational-algebra representation of a query specifies only **partially how to evaluate a query**; there are usually several ways to evaluate relational-algebra expressions.

Cont'd

As an illustration, consider the query:

- In SQL, a user wants to fetch the records of the instructor whose salary is greater than or equal to 75000.

```
select salary
from instructor
where salary < 75000;
```

- This query can be translated into either of the following relational-algebra expressions:
  - $\sigma_{salary<75000}(\Pi_{salary}(instructor))$
  - $\Pi_{salary}(\sigma_{salary<75000}(instructor))$

Cont'd

- Further, we can execute each relational-algebra ***operation*** by one of several different algorithms.
- For example, to implement the preceding selection, we can search every tuple in instructor to find tuples with salary less than 75000. If a B+-tree index is available on the attribute salary, we can use the index instead to locate the tuples.
- To specify fully **how to evaluate a query**, we need not only to provide the ***relational algebra expression***, but also to annotate it with ***instructions specifying how to evaluate each operation***.
- Annotations may state the algorithm to be used for a specific operation or the particular index or indices to use.

Cont'd

- A relational-algebra operation annotated with instructions on how to evaluate it is called **an evaluation primitive**.
- A sequence of primitive operations that can be used to evaluate a query is a **query-execution plan or query-evaluation plan.**
- Figure 15.2 illustrates an **evaluation plan** for our example query, in which a particular index (denoted in the figure as "index 1") is specified for the selection operation.

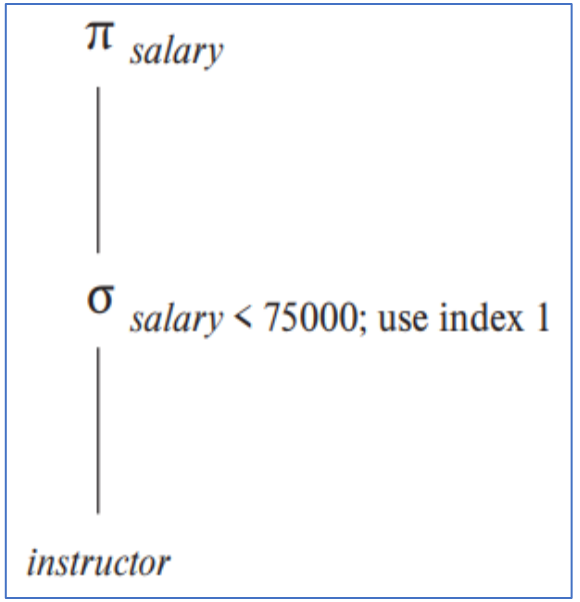


Figure 15.2 A query-evaluation plan.

Cont'd

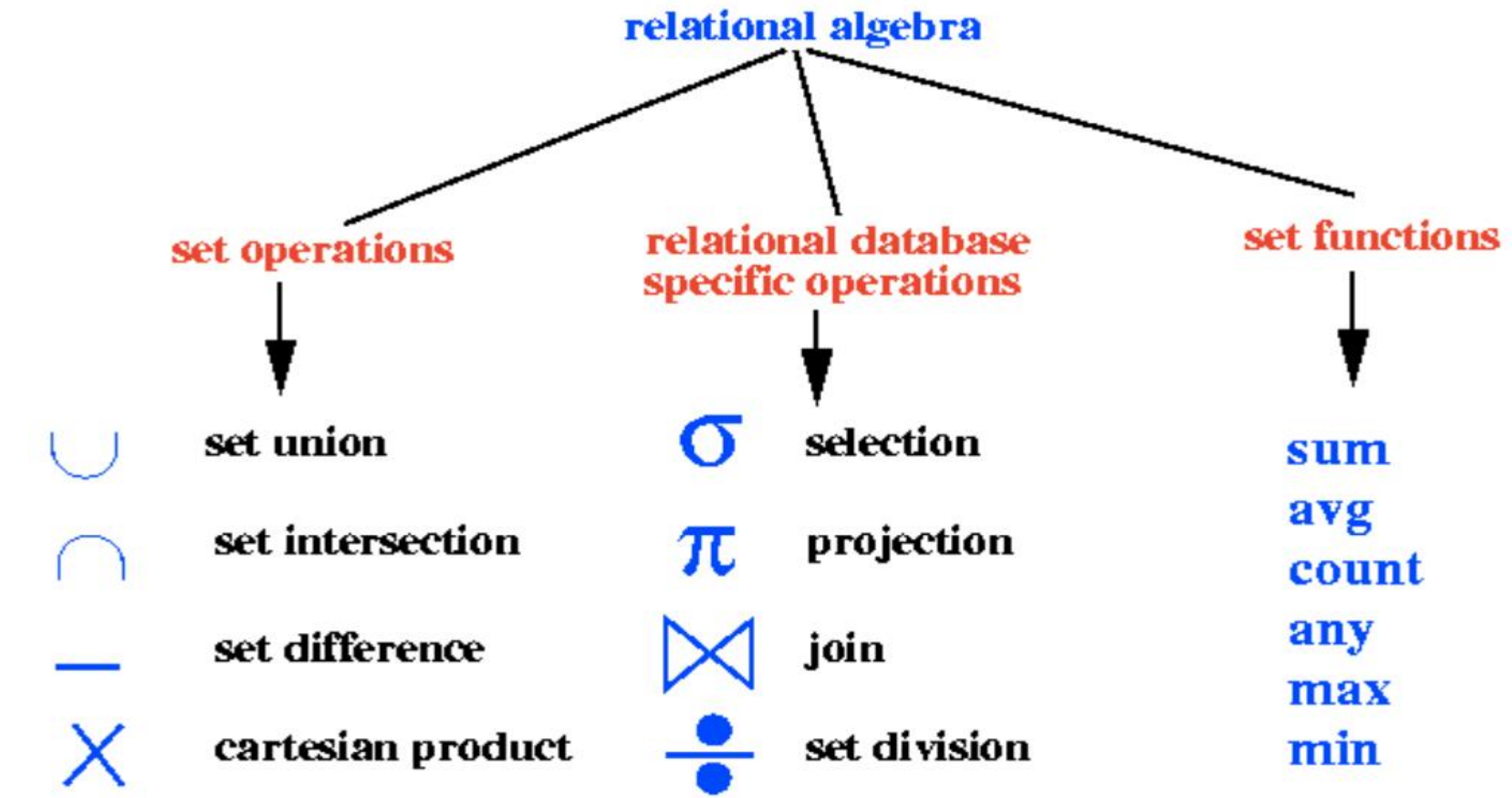
- The cost of the **query evaluation** can vary for different types of queries.
- We do not expect users to write their queries in a way that suggests the most efficient evaluation plan.
- Rather, it is the responsibility of the database system to construct **a query-evaluation plan** that minimizes the cost of query evaluation/execution; this task is called **query optimization**.
- For optimizing a query, the query optimizer should have an estimated cost analysis of each operation.

Cont'd

Query execution engine

- Once the **query plan** is chosen, the query is evaluated with that plan, and the result of the query is output.
  - *That means, a **query execution engine** is responsible for generating the output of the given query. It takes the query execution plan, executes it, and finally makes the output for the user query.*

Recap: Relational Algebra



Recap: SQL Query to RA

- How do you represent queries in RA?

- Database: Students(sid, sname, gpa)
  People(ssn, pname, address)

- SQL query:

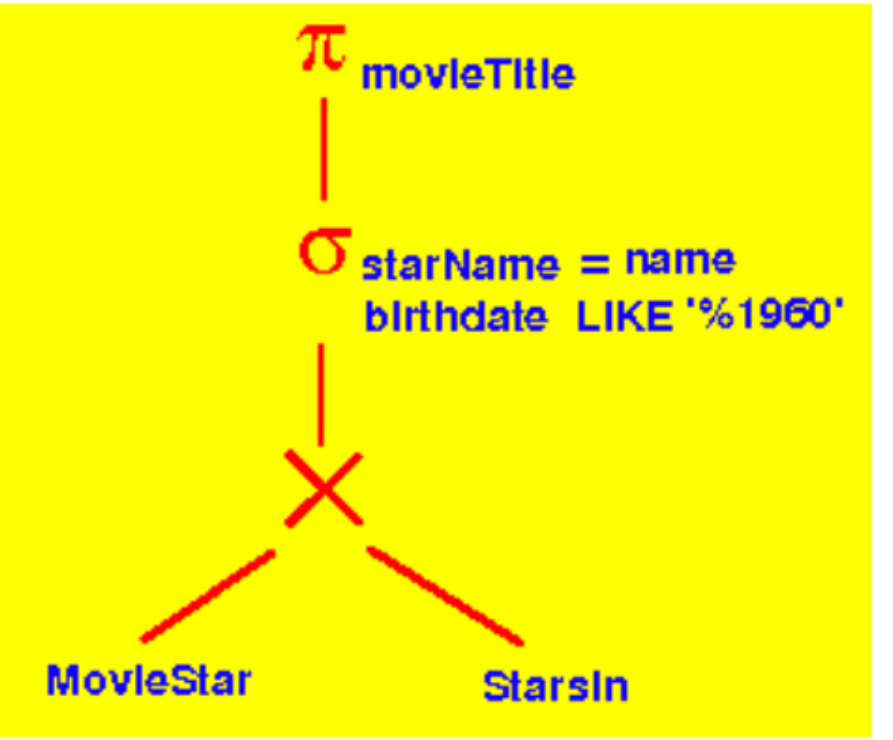
```
SELECT DISTINCT gpa, address
FROM   Students, People
WHERE  gpa >  3.5 AND sname = pname;
```

- RA query:

$$\pi_{\text{gpa,address}}(\sigma_{\text{gpa}>3.5}(\text{Students} \bowtie_{\text{sname}=\text{name}} \text{People}))$$

Query Tree (Plan)

- A tree data structure that corresponds to a relational algebra expression
  - Leaf nodes = input relations
  - Internal nodes = RA operations
- Execution of query tree
  - Start at the leaf nodes
  - Execute internal node whenever its operands are available



Query Decomposition(Parsing, validation, Translation)

1. **Parsing:**
   - Lexical Analysis, Syntax Analysis: semantics Analysis:
2. **Validation:**
   - **Authorization Checks:** The system verifies that the user has the necessary privileges to access the specified tables and perform the requested operations.
   - **Data Type Compatibility:** The data types of operands in expressions and comparisons are checked for compatibility.

3. Translation:

- The parsed and validated query is transformed into a RA expression.

Query Optimization

- Activity of choosing an efficient **execution strategy** for processing query.
  - ✓ As there are many equivalent transformations of same high-level query, and the role of **optimizer** is to select an efficient way to execute a query (choose one that minimizes resource usage, reduce total execution time of query).
- Therefore, the **optimizer** needs to:
  - ✓ determine different ways (plans) that one can execute a query,
  - ✓ determine the execution cost of each plan, and then
  - ✓ choose the most cost effective plan for execution.

Query Optimization

- The cost of query evaluation/execution can be measured in terms of a number of different resources, including
  - ✓ Number of disk accesses,
  - ✓ CPU time to execute a query, and,
  - ✓ In parallel and distributed database systems, the cost of communication.
- This must be taken into consideration to estimate cost of a plan. Disk accesses are usually the most dominating cost factor.

As an illustration, consider the query:

- Example: Find all managers who work in a London branch
- SELECT *FROM Staff s, Branch b WHEREs.branchNo = b.branchNo AND s.position = ‘Manager’ AND b.city = ‘london’;
- Results in these equivalent relational algebra statements

Three equivalent RA queries are:

1) σ(position=‘Manager’)^(city=‘London’)^(Staff.branchNo=Branch.branchNo)(**Staff X Branch**)

2) σ(position=‘Manager’)^(city=‘London’) (**Staff** $_{\text{Staff.branchNo = Branch.branchNo}}$ **Branch**)

3) (σ position='Manager'(**Staff**)) Staff.branchNo = Branch.branchNo (σ city='London’(**Branch**))

StaffID	Name	Position	BranchNo
1	Alice	Manager	10
2	Bob	Clerk	20
3	Charlie	Manager	10

BranchNo	City
10	London
20	New York

Cont'd

- Assume:
  - 1000 tuples in Staff; 50 tuples in Branch;
  - 50 Managers; 5 London branches;
  - no indexes or sort keys;
  - results of any intermediate operations stored on disk;
  - cost of the final write is ignored;
  - tuples are accessed one at a time.

Cont'd

Analysis of each Query Expression

- ***Analysis 1:***
  - i. *read each tuple from the two relations →n+m reads*
  - ii. *create a table of the Cartesian product→ nXm writes*
  - iii. *test each tuple of step 2→nXm read*
- ***Total No. of Disk access: →2(nXm) +n+m***
- ***Analysis 2:***
  - i. *read each tuple from the two relations →n+m reads*
  - ii. *create a table of the Join→ n writes*
  - iii. *test each tuple of step 2→n read*
- ***Total No. of Disk access: →3(n) +m***

- ***Analysis 3:***
  - i. *read each tuple from the two relations →n+m reads*
  - ii. *create a table of the Join→ n writes*
  - iii. *test each tuple of step 2→n read*
- ***Total No. of Disk access: →3(n) +m***

Example - Cost Comparison

- **Cost (in disk accesses) are:**

(1) (1000 + 50) + 2*(1000 * 50) = 101 050

(2) 2*1000 + (1000 + 50) = 3 050

(3) 1000 + 2*50 + 5 + (50 + 5) = 1 160

Cartesian product and join operations much more expensive than selection, and third option significantly reduces size of relations being joined together.

Approaches to Query Optimization

Using Heuristic in Query Optimization

Apply **heuristic rules** on standard initial query tree to find optimized equivalent query tree.

Main heuristic is to apply first the operations that reduce the size of intermediate results.

- E.g., Apply SELECT and PROJECT operations before applying the JOIN or other binary operations.
- • Apply more selective SELECT and join first

Transformation Rules for RA Operations

- **Relational algebra** is a powerful tool for expressing queries on relational databases.
- Transformation rules allow us to rewrite relational algebra expressions into equivalent forms, which can be crucial for query optimization.

Transformation Rules for RA Operations

Relational Algebraic Equivalence Transformation Rule

❖**Selection cascade: conjunctive selection condition can be broken/deconstructed into sequence of individual operations**

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

Example: $\boldsymbol{\sigma}_{\text{bno='B3'} \wedge} \boldsymbol{\sigma}_{\text{salary>15000}}(\text{Staff}) = \boldsymbol{\sigma}_{\text{bno='B3'}}(\boldsymbol{\sigma}_{\text{salary>15000}}(\text{Staff}))$

❖ **Commutativity of selection:**

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

Example: $\boldsymbol{\sigma}_{\text{bno='B3'}}(\boldsymbol{\sigma}_{\text{salary>15000}}(\text{Staff})) = \boldsymbol{\sigma}_{\text{salary>15000}}(\boldsymbol{\sigma}_{\text{bno='B3'}}(\text{Staff}))$

Transformation Rules for RA Operations

- **Cascade(sequence) of π: Only the final operations in a sequence of projection operations is needed, the others can be omitted; cascade of Π.**

$$\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{L_n}(E))\ldots)) = \Pi_{L_1}(E)$$

- **Example**: $\pi_{\text{plname}}(\pi_{\text{Pbno}}(\text{Staff})) = \pi_{\text{Plname}}(\text{Staff})$
- **Explanation**: A cascade or a series of projections is meaningless. This is because in the end, we are only selecting those columns which are specified in the last, or the outermost projection. Hence, it is better to collapse all the projections into just one i.e. the outermost projection.

Transformation Rules for RA Operations

❖Theta join operations are commutative:

$$E_1 \bowtie_\theta E_2 = E_2 \bowtie_\theta E_1$$

A theta join (denoted ⋈θ) is a join operation that combines tuples from two relations E1 and E2 based on a specified condition θ

- **Example**:
  - Let E_1 contain tuples $(1,'Alice')$ and $(2,'Bob')$ with attributes A and B.
  - Let E_2 contain tuples $(1,'Engineer')$ and $(3,'Doctor')$ with attributes C and D.
  - If we perform a theta join based on the condition $A = C$:

$$E_1 \bowtie_{A=C} E_2 \quad \text{results in} \quad \{(1,'Alice',1,'Engineer')\}$$

  - Conversely, performing the join as $E_2 \bowtie_{C=A} E_1$ will yield the same result:

$$E_2 \bowtie_{C=A} E_1 \quad \text{results in} \quad \{(1,'Engineer',1,'Alice')\}$$

Transformation Rules for RA Operations

❖**Selections can be combined with Cartesian products and theta joins:**

$$\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$$

$$\sigma_{\theta_1}(E_1 \bowtie_{\sigma_{\theta_2}} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$$

- When you perform a selection on the result of a Cartesian product of two relations E_1 and E_2 using a condition θ, it is equivalent to directly performing a theta join on those relations.

- When you perform a selection on the result of a theta join, this can be simplified to a single theta join that combines both selection conditions.
- The left-hand side, $\sigma_{\theta_1}(E_1 \bowtie \sigma_{\theta_2}(E_2))$, applies the selection θ_2 to E_2 first, then performs the join, and finally applies θ_1 to the result.
- The right-hand side, $E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$, directly combines the two conditions θ_1 and θ_2 in the join operation.

Transformation Rules for RA Operations

- **Natural-join operations are associative:**

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

- **Theta joins are associative in the following manner**

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ2 involves attributes from E2 and E3 only.

Transformation Rules for RA Operations

- The set operations union and intersection are commutative.

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

- Set difference is not commutative.
- Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

Transformation Rules for RA Operations

- **The selection operation distributes over the union, intersection, and set-difference operations.**

$$\sigma_P(E_1 - E_2) = \sigma_P(E_1) - E_2 = \sigma_P(E_1) - \sigma_P(E_2)$$

- **The projection operation distributes over the union operation.**

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

Pipelining

- **Materialization -** output of one operation is stored in temporary relation for processing by next. (heuristic approach).
- Could also pipeline results of one operation to another without creating temporary relation.
- Known as **pipelining** or **on-the-fly processing**.
- Pipelining can **save on cost of creating temporary relations** and reading results back in again.
- Generally, pipeline is implemented as separate process or thread.

Materialization

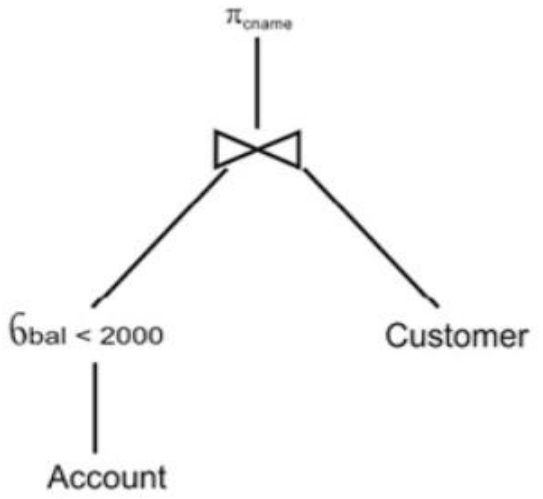
- Its a process of storing output of an operator operation in the temporary relation for processing by the next operator.
- It starts from lowest level operation in the expression, which are at the bottom of the query tree.

Consider the below relational algebra query as

$\Pi_{cname}\ (\sigma_{bal<2000}(account) \bowtie customer))$

The pictorial representation of the above query as shown below.

Main problem with this approach is the need to construct temporary relation which must be written to disk.



Pipelining

- Process of passing result of one relational operator to another operator directly without storing it in temporary relation.
- Pipelining is used to improve performance of the queries.
- Pipeline operation eliminates the cost of reading and writing temporary relations.
- The efficiency of the query evaluation can be improved by reducing the number of temporary files produced.
- A buffer is created for each pair of adjacent operation to hold tuples being passed from 1st operation to second one.

।। विद्या धनम् सर्वधन: प्रधानम् ।।

Example of Pipelining

$$\sigma_{bal < 2500} (\Pi_{bal} (acc))$$


Thank You!